

AhamX Bodhi

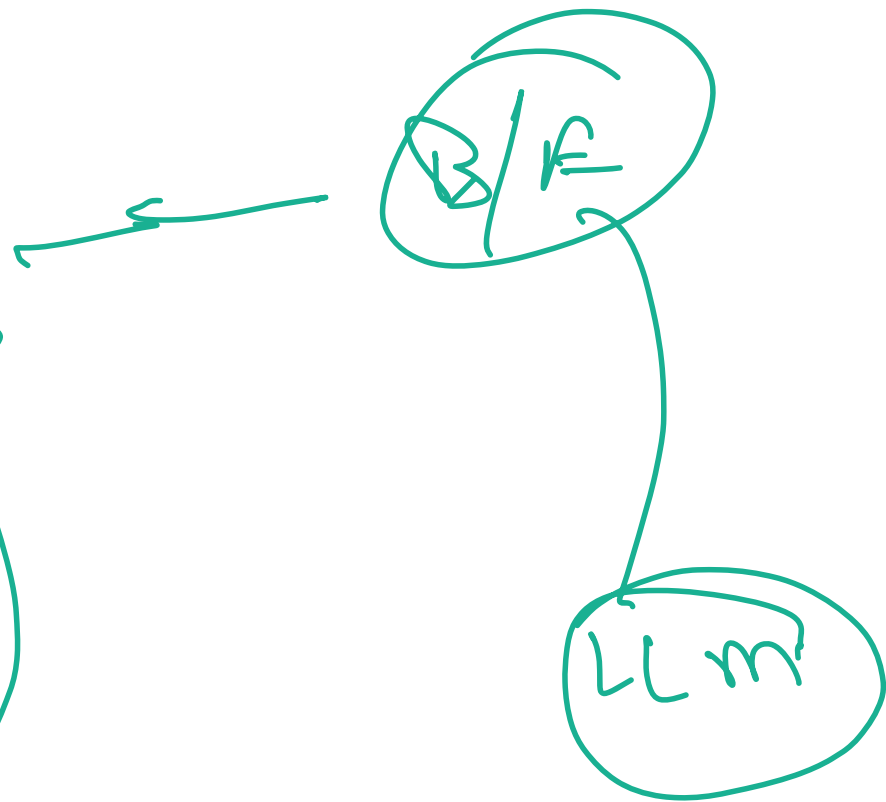
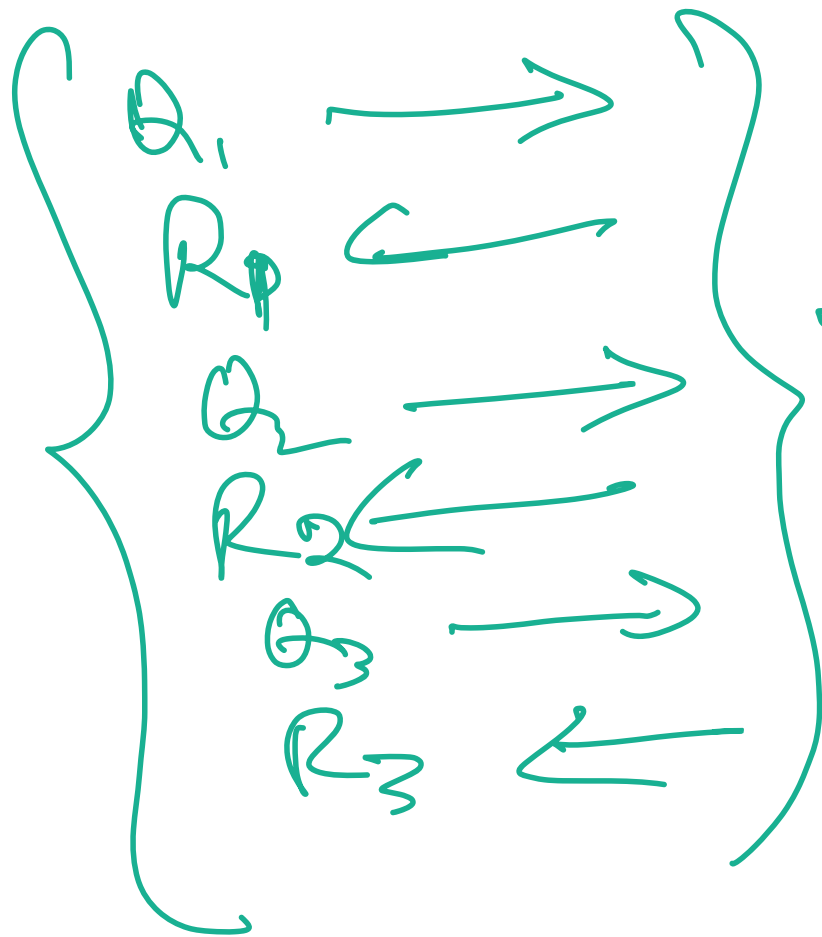
Where I Becomes Infinite

Repetition Penalty

Module 3.1: Working with LLM APIs

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026

FF





Session Overview

Prerequisites

- How LLMs generate text token by token
- Logits and the softmax probability distribution
- Temperature, top-k, and top-p parameters
- Autoregressive generation and context window

What You Will Learn

- Why LLMs fall into repetition loops
- How repetition penalty modifies logits
- The difference between repetition and presence penalty
- Frequency penalty as an alternative mechanism
- Practical values and when to apply each



Why LLMs Fall Into Repetition Loops

The Autoregressive Feedback Problem

Language models predict each token conditioned on all previous tokens. Once a phrase or sentence pattern appears in the context, the model **recognises that pattern as likely** and assigns high probability to repeating it. This creates a positive feedback loop: the model repeats itself, which increases the probability of repeating again, which leads to even more repetition.

When Repetition is Most Common

- **Low temperature / greedy decoding:** The model always picks the highest-probability token; if a phrase is likely once, it remains likely and the same phrase recurs indefinitely
- **Long outputs:** With more tokens generated, drift and repetition accumulate; the model “forgets” it already said something
- **Smaller models:** Limited capacity to maintain coherent topic diversity over extended text; more prone to looping
- **Weak instruction following:** Model does not strongly honour “do not repeat yourself” in the system prompt



Repetition Penalty --- The Core Mechanism

How Repetition Penalty Modifies Logits

Before computing probabilities, the logit z_i for any token i that has already appeared in the context is **divided** by the penalty factor $\theta \geq 1.0$:

$$z'_i = \begin{cases} z_i / \theta & \text{if token } i \text{ appears in the context} \\ z_i & \text{otherwise} \end{cases}$$

- $\theta = 1.0$: No effect (identity operation)
- $\theta > 1.0$: Penalises previously seen tokens
- For *positive* logits: division reduces the logit (lower probability)
- For *negative* logits: division brings logit closer to zero (counterintuitively increases probability for negative logits — a known quirk of this formulation)
- Typical range: $\theta \in [1.0, 1.5]$: values above 1.5 often cause avoidance of grammatically necessary words

1.2-1.5



Presence Penalty and Frequency Penalty

Presence Penalty (OpenAI)

Adds a fixed negative bias α to the logit of any token that has appeared *at least once* in the context, regardless of how many times:

$$z'_i = z_i - \alpha \cdot \mathbf{1}[\text{token appeared}]$$

- Discourages re-introduction of topics
- Range: -2.0 to $+2.0$
- Positive values $\rightarrow$ avoidance; negative $\rightarrow$ promotion
- Constant penalty: repeated and once-used tokens are treated equally

Frequency Penalty (OpenAI)

Adds a penalty proportional to the *number of times* a token has appeared in the context:

$$z'_i = z_i - \beta \cdot \text{count}(i)$$

- Discourages tokens used *many* times more than tokens used once
- Proportional: more appearances $\rightarrow$ stronger penalty
- Range: -2.0 to $+2.0$
- Better for reducing “filler” words and phrases that creep in at high frequency

Comparing Repetition Control Mechanisms



Property	Repetition Penalty	Presence Penalty	Frequency Penalty
Operation	Division of logit	Additive constant	Additive, proportional to count
Scales with count?	No (binary: seen/unseen)	No (binary)	Yes (proportional)
API	Hugging Face, llama.cpp	OpenAI, Anthropic	OpenAI, Anthropic
Neutral value	1.0	0.0	0.0
Promotes novelty?	Yes	Yes (new topics)	Yes (diverse vocabulary)
Typical range	1.0 – 1.5	0.0 – 1.5	0.0 – 1.5
Main weakness	Penalises common function words	Ignores frequency	May over-penalise common words



Side Effects of Repetition Penalty

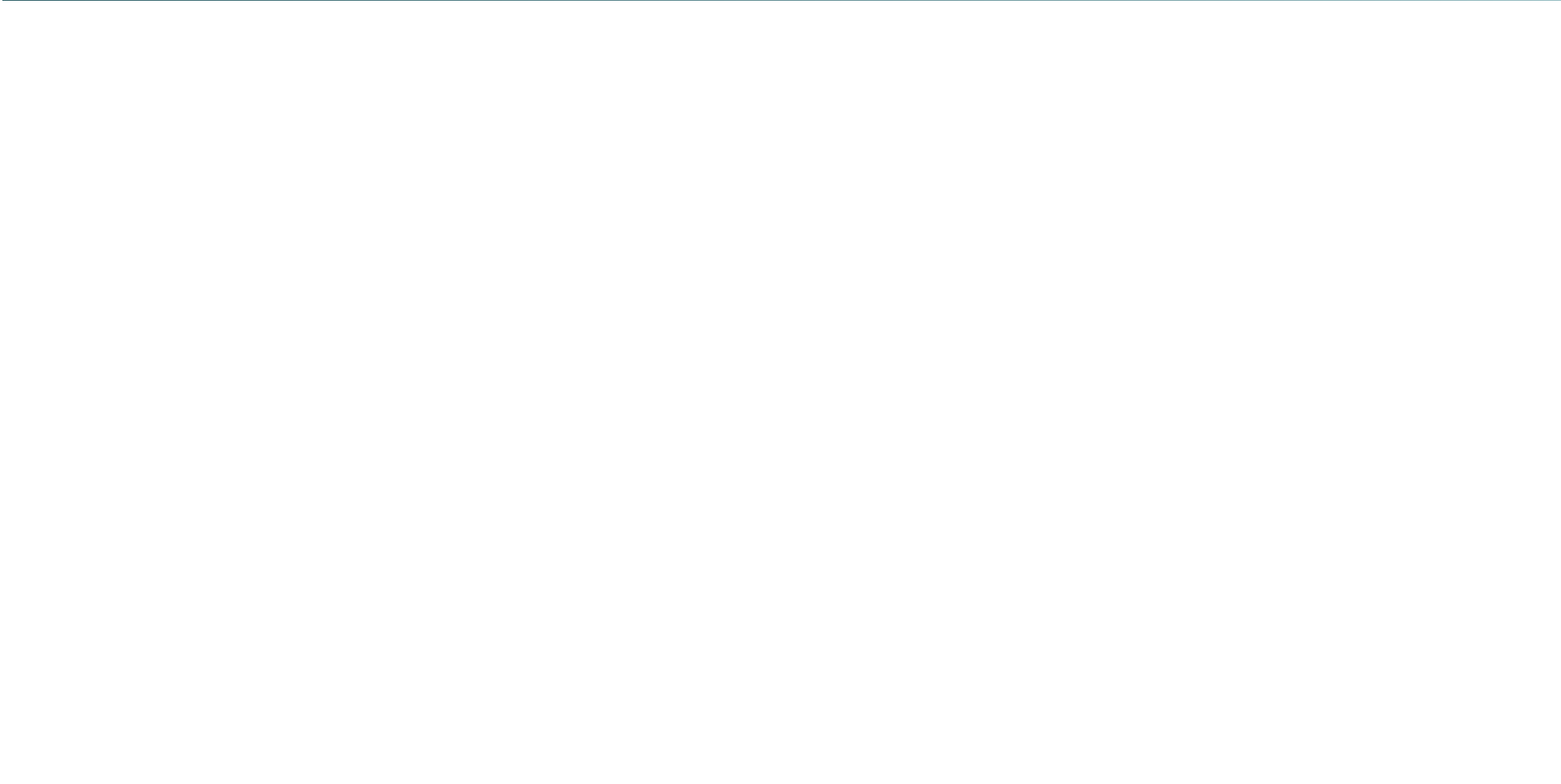
What Can Go Wrong with Excessive Penalty

- **Avoidance of function words:** Common words like "the", "is", "a" appear constantly in text. A high penalty eventually reduces their probability so much the model avoids them, breaking grammatical correctness
- **Incoherent topic switching:** A penalty that is too strong forces the model to constantly introduce new vocabulary, causing it to change subject mid-paragraph
- **Proper nouns and names:** Entity names that must be repeated (character names in a story, a product name in a description) are incorrectly penalised along with unwanted repetitions
- **Poetry and structured text:** Intentional repetition (refrains, anaphora) is broken by any non-zero repetition penalty
- **Code:** Variable names and keywords must repeat; penalty causes incorrect, non-compilable code with random name variation

Practical Tuning Reference



Use Case	Rep. Penalty	Freq. Penalty	Notes
Code generation	1.0 (off)	0.0	Keywords must repeat freely
Poetry / verse	1.0 (off)	0.0	Intentional repetition must be allowed
Factual Q&A	1.0 – 1.1	0.0 – 0.1	Light penalty to avoid redundancy
Long-form text	1.1 – 1.2	0.1 – 0.3	Prevent phrase loops in essays
Creative stories	1.1 – 1.3	0.2 – 0.5	Encourage vocabulary diversity
Chat/dialogue	1.0 – 1.15	0.0 – 0.2	Mild; don't break pronoun references
Summarisation	1.1 – 1.2	0.1 – 0.2	Avoid repeating source phrases





GenAI Application: Long-Form Content Platforms

Repetition Control in Content Generation Tools

- **Jasper AI:** Applies frequency_penalty=0.3–0.5 for blog post and marketing copy generation. Without penalty, Jasper's outputs would repeat key phrases every paragraph — visible quality degradation in real user feedback
- **Copy.ai:** Uses presence_penalty to prevent the same product benefit from being restated in every paragraph of a landing page draft
- **Sudowrite:** Exposes repetition penalty as a “freshness” slider in the UI. Higher freshness maps to higher penalty; users who want thematic consistency lower the slider
- **Notion AI:** Long document continuation uses frequency penalty to prevent key-term repetition while allowing structured headers (which do intentionally repeat document sections)

Content quality at scale depends on careful repetition penalty calibration.



GenAI Application: Chatbots and Conversational AI

Why Chatbots Need Repetition Control

- Without penalty, chatbots often repeat the user's question back in every response
- "I understand you are asking about..." prefix recurs in every turn — frustrating in chat
- Multi-turn history grows the context; the model sees its own prior responses and re-uses phrases
- Presence penalty ≈ 0.1 – 0.2 keeps responses feeling fresh across conversation turns

Implementation in Production

- Many production chatbots apply penalty only over a *sliding window* of recent tokens rather than the full context, to avoid penalising function words too heavily in long conversations
- Some systems selectively exempt entity names from the penalty using a whitelist approach
- LangChain memory modules can optionally summarise old context, reducing the token space over which penalties accumulate



GenAI Application: Open-Source LLM Serving

Repetition Penalty in Local and Self-Hosted Models

- **llama.cpp / Ollama:** Expose `repeat_penalty` (default 1.1) and `repeat_last_n` (window size, default 64 tokens). The window size is critical: penalising only the last 64 tokens avoids over-penalising common function words that appear throughout a long document
- **Text Generation WebUI (AUTOMATIC1111 fork):** Provides a full repetition control panel including penalty, window size, and a “no-repeat n-gram” option that hard-blocks repeated n-grams of configurable length
- **Hugging Face** `generate()`: The `repetition_penalty` parameter and `no_repeat_ngram_size` parameter work independently; production pipelines often use both together
- **vLLM:** Passes repetition penalty through to the sampling kernel; compatible with the OpenAI-style API `frequency_penalty` parameter for drop-in compatibility

GenAI Application: Summarisation and Extraction



Extractive Repetition Risk

- Summarisation models condition on the source document; source phrases are already in context before generation
- Without penalty: the model parrots verbatim sentences from the source rather than paraphrasing
- Frequency penalty 0.2–0.4 encourages the model to reformulate rather than copy
- GPT-4 summarisation prompts often include explicit “do not repeat verbatim” instructions alongside low frequency penalty as a two-layer defence

Legal and Compliance Documents

- Contract summarisation: low penalty (≤ 0.1) because legal terms must be reproduced exactly
- Medical note summarisation: near-zero penalty to preserve exact clinical terminology
- News summarisation: moderate frequency penalty (0.3) for readable, non-verbatim summaries
- Academic abstract generation: presence penalty to avoid restating the paper title in every sentence

Session Summary



Key Takeaways

1. **Repetition** in LLMs is caused by the autoregressive feedback loop: previously generated tokens increase the probability of re-generating the same tokens
2. **Repetition penalty** divides logits of seen tokens by $\theta \geq 1.0$ before softmax; neutral value is 1.0
3. **Presence penalty** adds a fixed additive bias against any previously seen token (binary: seen/unseen)
4. **Frequency penalty** penalises tokens proportionally to their count in context (gradual accumulation)
5. Excessive penalty breaks grammar, proper nouns, and code; always validate outputs before deploying penalty settings
6. Match the penalty mechanism to the task: off for code/poetry, light for chat, moderate for long-form and creative writing

AhamX Bodhi

Where I Becomes Infinite